Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №1

по дисциплине «Информационные системы»

Вариант № 11337

Выполнил:
Гришин Артем Евгеньевич
P3311

Проверил:
Тюрин Иван Николаевич

Санкт-Петербург
2026

Содержание

1 Задание **2**

2 UML-диаграммы классов и пакетов **5**

2.1 Диаграмма классов 5

2.2 Диаграмма пакетов 7

3 Исходный код **8**

4 Описание решения **8**

4.1 Архитектура 8

4.2 Хранение и ограничения полей 8

4.3 Валидация и сообщения об ошибках 9

4.4 Таблица: пагинация, фильтрация и сортировка 9

4.5 Удаление координат с переприв язкой 10

4.6 Синхронизация клиентов 10

4.7 Специальные операции 11

4.8 Клиентская часть 12

4.9 Сборка и развёртывание 12

5 Выводы **13**

1. Задание

Реализовать информационную систему, которая позволяет взаимодействовать с объектами класса `Vehicle`, описание которого приведено ниже:

```
public class Vehicle {
    private long id; // Значение поля должно быть больше 0,
                     // уникальным и генерироваться автоматически
    private String name; // Поле не может быть null, строка не может
     ↪  быть пустой
    private Coordinates coordinates; // Поле не может быть null
    private java.util.Date creationDate; // Поле не может быть null,
                                         // значение генерируется
                                          ↪  автоматически
    private VehicleType type; // Поле не может быть null
    private int enginePower; // Значение поля должно быть больше 0
    private int numberOfWheels; // Значение поля должно быть больше
     ↪  0
    private long capacity; // Значение поля должно быть больше 0
    private Float distanceTravelled; // Поле может быть null,
                                     // значение должно быть больше
                                      ↪  0
    private int fuelConsumption; // Значение поля должно быть больше
     ↪  0
    private FuelType fuelType; // Поле не может быть null
}

public class Coordinates {
    private Double x; // Значение поля должно быть больше -415,
                      // поле не может быть null
    private Float y; // Максимальное значение поля: 663,
                     // поле не может быть null
}

public enum VehicleType {
    CAR,
    SUBMARINE,
    BICYCLE,
    HOVERBOARD;
}

public enum FuelType {
    ELECTRICITY,
    DIESEL,
    ANTIMATTER;
```

```
}
```

Разработанная система должна удовлетворять следующим требованиям:

- Основное назначение информационной системы — управление объектами, созданными на основе заданного в варианте класса.
- Необходимо, чтобы с помощью системы можно было выполнить следующие операции с объектами: создание нового объекта, получение информации об объекте по ИД, обновление объекта (модификация его атрибутов), удаление объекта. Операции должны осуществляться в отдельных окнах (интерфейсах) приложения. При получении информации об объекте класса должна также выводиться информация о связанных с ним объектах.
- При создании объекта класса необходимо дать пользователю возможность связать новый объект с объектами вспомогательных классов, которые могут быть связаны с созданным объектом и уже есть в системе.
- Выполнение операций по управлению объектами должно осуществляться на серверной части (не на клиенте), изменения должны синхронизироваться с базой данных.
- На главном экране системы должен выводиться список текущих объектов в виде таблицы (каждый атрибут объекта — отдельная колонка в таблице). При отображении таблицы должна использоваться пагинация, если все объекты не помещаются на одном экране.
- Нужно обеспечить возможность фильтровать/сортировать строки таблицы, которые показывают объекты, по значениям строковых колонок. Фильтрация элементов должна производиться по неполному совпадению.
- Переход к обновлению (модификации) объекта должен быть возможен из таблицы с общим списком объектов и из области с визуализацией объекта (при ее реализации).
- При добавлении/удалении/изменении объекта он должен автоматически появиться/исчезнуть/измениться в интерфейсах у других пользователей, авторизованных в системе.
- Если при удалении объекта с ним связан другой объект, связанные объекты должны быть связаны с другим объектом (по выбору пользователя), а изначальный объект удален.

- Пользователи должны иметь возможность просмотра всех объектов. Для модификации объекта должно открываться отдельное диалоговое окно. При вводе некорректных значений в поля объекта должны появляться информативные сообщения о соответствующих ошибках.

В системе должен быть реализован отдельный пользовательский интерфейс для выполнения специальных операций над объектами:

- Удалить все объекты, значение поля `fuelType` которого эквивалентно заданному.
- Вернуть один (любой) объект, значение поля `creationDate` которого является минимальным.
- Вернуть массив объектов, значение поля `fuelType` которых больше заданного.
- Найти все транспортные средства с мощностью двигателя в заданном диапазоне.
- Добавить транспортному средству с заданным id указанное число колёс.
- Представленные операции должны быть реализованы в рамках компонентов бизнес-логики приложения без прямого использования функций и процедур БД.

Особенности хранения объектов, которые должны быть реализованы в системе:

- Организовать хранение данных об объектах в реляционной СУБД (PostgreSQL). Каждый объект, с которым работает ИС, должен быть сохранён в базе данных.
- Все требования к полям класса (указанные в виде комментариев к описанию классов) должны быть выполнены на уровне ORM и БД.
- Для генерации поля id использовать средства базы данных.
- Для подключения к БД на кафедральном сервере использовать хост `pg`, имя базы данных `studs`, имя пользователя/пароль совпадают с таковыми для подключения к серверу.

При создании системы нужно учитывать следующие особенности организации взаимодействия с пользователем:

- Система должна реагировать на некорректный пользовательский ввод, ограничивая ввод недопустимых значений и информируя пользователей о причине ошибки.
- Переходы между различными логически обособленными частями системы должны осуществляться с помощью меню.
- При добавлении/удалении/изменении объекта он должен автоматически появиться/исчезнуть/измениться в области у всех других клиентов.

При разработке ИС должны учитываться следующие требования:

- В качестве основы для реализации ИС необходимо использовать Jakarta EE, CDI beans.
- Для создания уровня хранения необходимо использовать JPA + EclipseLink.
- Разные уровни приложения должны быть отделены друг от друга, разные логические части ИС должны находиться в отдельных компонентах.

2. UML-диаграммы классов и пакетов

2.1. Диаграмма классов

На диаграмме классов (рис. 1) слои приложения идут сверху вниз: REST-ресурсы, сервисы, репозитории и сущности, а сбоку — классы WebSocket. Сплошные стрелки означают, что класс хранит ссылку на другой класс (внедрён через `@Inject`), пунктирные — зависимость: репозиторий работает с сущностью через JPQL, сервис отправляет CDI-событие, которое ловит `ChangeNotifier`.

Сверху показаны классы платформы Jakarta, к которым подключается приложение: `RestApplication` наследует `Application`, а три маппера реализуют `ExceptionMapper`, и каждый связан с исключением, которое он обрабатывает. Напрямую мапперы никто не вызывает: сервис бросает исключение, а JAX-RS сам находит подходящий `@Provider` по типу исключения.

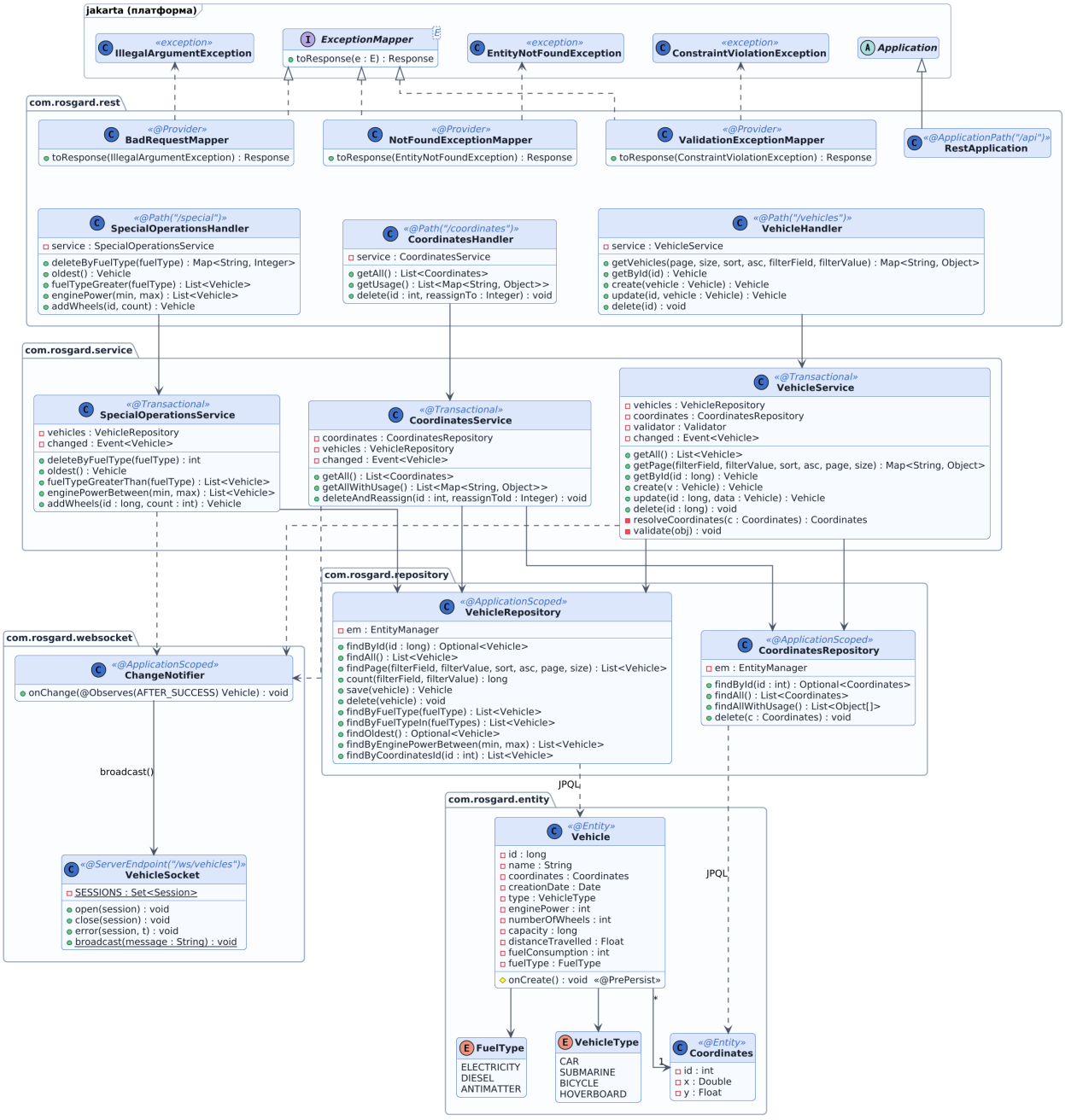


Рис. 1: Диаграмма классов

2.2. Диаграмма пакетов

Серверный код разделён на пакеты по слоям. Зависимости идут только сверху вниз: `rest` знает про `service`, `service` — про `repository`, и все они используют `entity`. Единственная связь «вбок» — от `service` к `websocket`, и она идёт через CDI-событие, так что сервисы не вызывают WebSocket напрямую. Клиентская часть обращается к серверу двумя путями: запросами к REST API и через WebSocket-соединение.

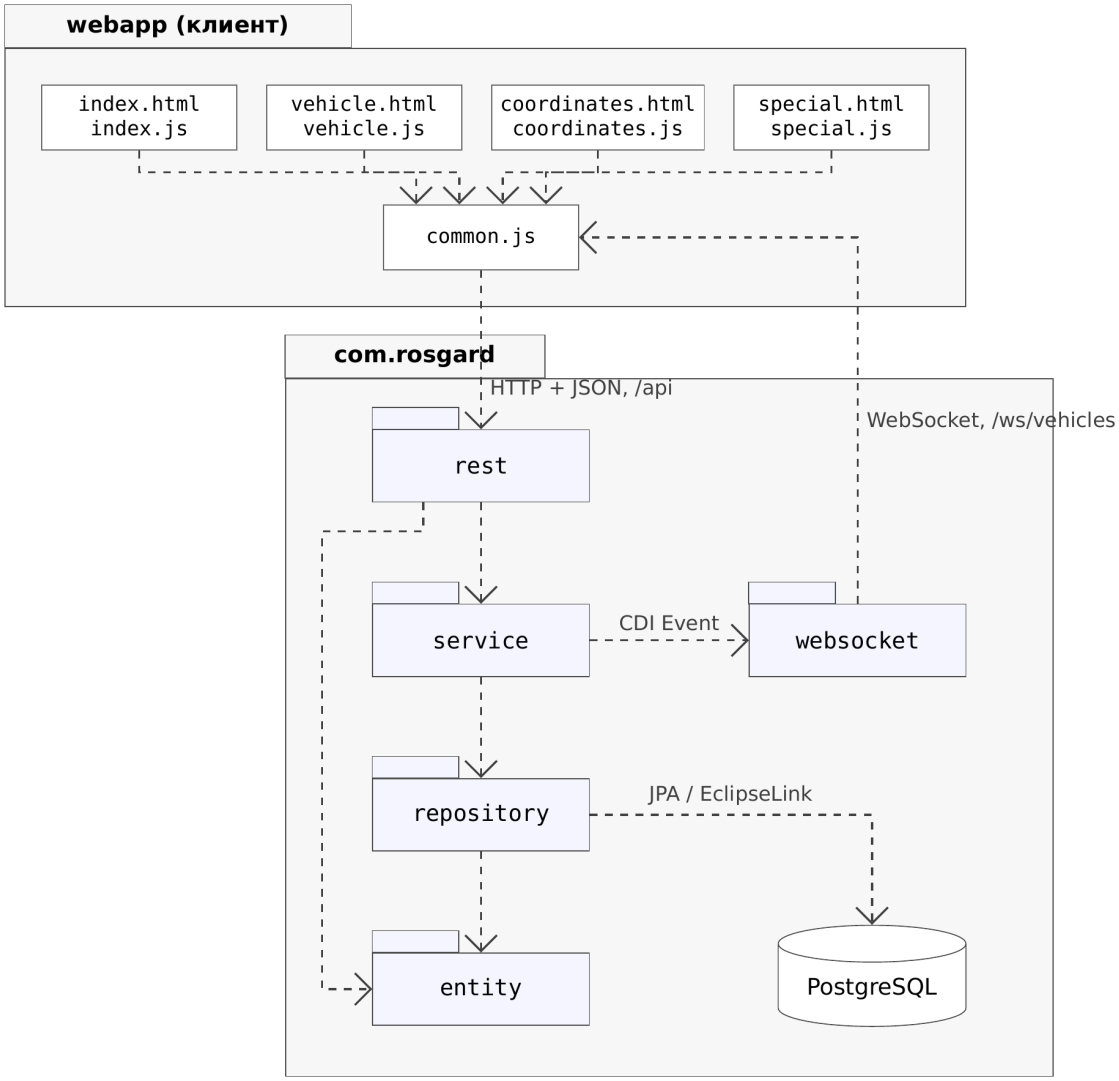


Рис. 2: Диаграмма пакетов

3. Исходный код

Исходный код лежит в репозитории `https://github.com/RosgardX/itmo-labs`, в каталоге `IS/IS1`. Код я выкладывал не одним коммитом, а цепочкой pull request'ов по слоям: модель, репозитории, сервисы, WebSocket, REST, интерфейс и отчёт. Каждая ветка создана от предыдущей, поэтому в каждом PR видно только то, что добавляет этот слой, и его можно проверить отдельно.

4. Описание решения

4.1. Архитектура

Приложение я разделил на слои, каждый в своём пакете:

- `entity` — классы предметной области, размеченные аннотациями JPA;
- `repository` — работа с базой через `EntityManager` и JPQL, больше ничего;
- `service` — бизнес-логика: проверки, транзакции, спецоперации;
- `rest` — тонкий слой, который принимает HTTP-запрос, вызывает сервис и отдаёт JSON;
- `websocket` — оповещение клиентов об изменениях.

Так у каждого класса одна задача. REST-ресурсы ничего не знают о базе, а репозитории — о HTTP. Если завтра понадобится, например, вызывать те же операции не из REST, а из другого места, сервисы можно переиспользовать без изменений. Все компоненты — CDI-бины с областью `@ApplicationScoped`, связи между ними задаются через `@Inject`.

4.2. Хранение и ограничения полей

По заданию все ограничения полей должны выполняться и на уровне ORM, и на уровне БД. Я сделал оба уровня явно. На стороне Java стоят аннотации Bean Validation (`@NotNull`, `@NotBlank`, `@Positive` и другие) с понятными сообщениями на русском. На стороне базы каждое поле описано через `columnDefinition` с `NOT NULL` и `CHECK`, например `integer NOT NULL CHECK (enginePower > 0)`.

Зачем два уровня: проверка в Java даёт пользователю нормальное сообщение об ошибке ещё до обращения к базе, а CHECK в базе гарантирует, что некорректная строка не попадёт в таблицу, даже если её вставят в обход приложения.

Идентификатор генерирует сама база: @GeneratedValue(strategy = IDENTITY) и тип bigserial. Дата создания проставляется автоматически в методе с @PrePersist, так что клиент не может её подменить.

Координаты я вынес в отдельную сущность Coordinates со связью @ManyToOne. Это нужно для требования «связать новый объект с уже существующими вспомогательными объектами»: несколько машин могут ссылаться на одни координаты, и при создании можно выбрать уже существующие координаты, а не вводить их заново.

Схему таблиц создаёт EclipseLink при деплое (schema-generation со значением create). Я выбрал create, а не drop-and-create, чтобы данные не стирались при каждом передеплое.

4.3. Валидация и сообщения об ошибках

Все проверки выполняются на сервере, в сервисах. Перед сохранением сервис вызывает Validator, и если есть нарушения, бросает ConstraintViolationException. Для случаев «объект не найден» и «неверный параметр» сервисы бросают EntityNotFoundException и IllegalArgumentException.

Превращать исключения в HTTP-ответы я поручил мапперам JAX-RS. Их три, по одному на тип ошибки, и каждый возвращает ответ с кодом 400 или 404 и телом вида {"errors": [...]}. Благодаря этому в REST-ресурсах нет ни одного try/catch, а клиент всегда получает ошибки в одном формате и просто выводит их список под формой.

На клиенте тоже есть ограничения ввода (required, min, max у полей формы), но это только для удобства: сервер всё равно проверяет данные сам.

4.4. Таблица: пагинация, фильтрация и сортировка

Пагинацию, фильтрацию и сортировку я сделал на сервере, а не в браузере. Если отдавать клиенту всю таблицу и листать её на месте, то при большом числе объектов страница будет грузиться всё дольше. Сервер же возвращает только нужную страницу (через setFirstResult и setMaxResults) и общее число найденных записей, чтобы клиент мог нарисовать номера страниц.

Фильтрация работает по неполному совпадению: по названию через LOWER(name) LIKE, по типу и типу топлива — по совпадению части назва-

ния значения перечисления.

Отдельно пришлось подумать о сортировке. Поле сортировки приходит от клиента строкой и подставляется в текст JPQL-запроса, а параметром запроса его передать нельзя. Чтобы через это поле нельзя было внедрить произвольный JPQL, я храню список разрешённых полей и отклоняю всё остальное:

```
if (!SORT_FIELDS.contains(sort)) throw new
 ↪  IllegalArgumentException("Нельзя сортировать по полю " + sort);
```

4.5. Удаление координат с перепривязкой

Если удалить координаты, на которые ссылаются машины, внешний ключ в базе не даст это сделать. По заданию пользователь должен выбрать, к каким другим координатам привязать эти машины. Поэтому на странице координат рядом с каждой записью показывается, сколько машин к ней привязано, а при удалении появляется выбор новых координат.

На сервере всё происходит в одной транзакции: сервис находит привязанные машины, переставляет им ссылку на выбранные координаты и только потом удаляет старые. Если что-то пойдёт не так, транзакция откатится целиком и машины не останутся без координат. Если пользователь не выбрал, куда перепривязать, или выбрал те же координаты, сервер вернёт понятную ошибку.

4.6. Синхронизация клиентов

Задание требует, чтобы изменения сразу появлялись у всех пользователей. Я рассматривал несколько вариантов:

1. Short polling — клиент раз в несколько секунд сам спрашивает сервер, не изменилось ли что-нибудь;
2. Long polling — клиент отправляет запрос, а сервер держит его открытым, пока не появятся изменения;
3. Server-Sent Events — сервер отправляет события по одному постоянному HTTP-соединению;
4. WebSocket — постоянное двустороннее соединение.

Short polling я отбросил сразу: он постоянно нагружает сервер и базу запросами, даже когда ничего не меняется, и всё равно даёт задержку. Long polling — по сути обходной путь, чтобы получить то же, что дают SSE

или WebSocket. Между SSE и WebSocket я выбрал WebSocket: он входит в Jakarta EE (`jakarta.websocket`) и уже есть в WildFly, сервер на нём пишется одним классом с `@ServerEndpoint`, а на клиенте хватает стандартного объекта `WebSocket` без библиотек. То, что WebSocket двусторонний, мне здесь не нужно, но и не мешает.

Важный момент — когда именно отправлять уведомление. Если отправить его сразу после изменения в сервисе, клиент может перезапросить данные раньше, чем транзакция закоммитится, и получить старое состояние. Поэтому сервисы только публикуют CDI-событие, а `ChangeNotifier` принимает его с фазой `AFTER_SUCCESS`, то есть уже после успешного коммита:

```
public void onChange(@Observes(during =
 ↪  TransactionPhase.AFTER_SUCCESS) Vehicle changed) {
    VehicleSocket.broadcast("vehicles");
}
```

Заодно так сервисы не зависят от WebSocket напрямую: они не знают, кто слушает событие.

В сообщении я передаю не сам изменённый объект, а только сигнал «данные изменились». У каждого пользователя своя страница таблицы, свой фильтр и своя сортировка, и по одному объекту клиент не может понять, попадает ли он на его страницу и куда. Проще и надёжнее при сигнале перезапросить текущую страницу с теми же параметрами. Если соединение оборвалось, клиент через пару секунд подключается заново.

4.7. Специальные операции

Все пять операций находятся в отдельном сервисе `SpecialOperationsService`, а вызываются они с отдельной страницы. По заданию их нельзя делать функциями и процедурами БД, поэтому логика написана на Java, а к базе идут только обычные JPQL-запросы через репозиторий:

- удаление по типу топлива: сервис находит все такие машины и удаляет их, а клиенту возвращает, сколько удалено;
- объект с минимальной датой создания: запрос с сортировкой по `creationDate` и ограничением в одну запись;
- объекты с типом топлива больше заданного: «больше» я понимаю как порядок значений в перечислении `FuelType`. Сервис сам вычисляет, какие значения идут после заданного, и ищет машины с любым из них;

- поиск по диапазону мощности: запрос с `BETWEEN`, а если минимум больше максимума, сервер возвращает ошибку;
- добавление колёс: сервис находит машину, проверяет, что число колёс положительное и сумма не переполнит `int`, и меняет поле. Отдельный `save` не нужен: объект управляется `EntityManager`, и изменение запишется в базу при коммите транзакции.

Операции, которые меняют данные, тоже публикуют событие, так что результат сразу виден у всех клиентов.

4.8. Клиентская часть

Интерфейс я написал на обычных HTML, CSS и JavaScript, без фреймворков. Страниц всего четыре, и сборщик или отдельный фреймворк для них был бы лишним. Все данные клиент получает от REST API и сам строит таблицу.

Общий код вынесен в `common.js`: меню для перехода между разделами (объекты, координаты, спецоперации), диалог создания и редактирования, вывод ошибок и подключение к WebSocket. Создание и редактирование открываются в отдельном диалоговом окне, причём редактировать можно и из таблицы, и со страницы просмотра объекта. В форме можно выбрать уже существующие координаты или ввести новые.

4.9. Сборка и развёртывание

Проект собирается Maven в `IS1.war`. Используются Java 17, Jakarta EE API 11 и EclipseLink 4.0.4. Для EclipseLink я подключил только модуль JPA (`org.eclipse.persistence.jpa`): полный артефакт тянет за собой MOXy, который подменял сериализацию JSON и ломал ответы с `Map` и `List`.

Приложение не содержит параметров подключения к базе: в `persistence.xml` указан только JNDI-ресурс `java:jboss/datasources/IS1`. Во время разработки я запускал его на локальном WildFly с локальной базой PostgreSQL, а для сдачи настроил такой же datasource на WildFly на сервере helios, подключённый к базе `studs` на хосте `pg`. Код приложения при этом менять не пришлось.

5. Выводы

В ходе работы я разработал информационную систему для управления объектами `Vehicle` на Jakarta EE. Приложение разделено на слои: сущности, репозитории, сервисы, REST API и WebSocket, и у каждого слоя своя ответственность.

Ограничения полей выполняются и на уровне ORM через Bean Validation, и на уровне базы через `NOT NULL` и `CHECK`. Ошибки обрабатываются централизованно мапперами JAX-RS и доходят до пользователя понятными сообщениями. Таблица поддерживает серверную пагинацию, фильтрацию и сортировку, а изменения сразу появляются у всех клиентов благодаря WebSocket и CDI-событиям, которые срабатывают после коммита транзакции.

Я разобрался, как устроены JPA и EclipseLink, как CDI связывает компоненты и доставляет события, как JAX-RS сам вызывает мапперы исключений и почему уведомлять клиентов нужно только после коммита.